

Node.js Interview Questions & Answers - Complete Guide (50+ Questions)

Section 1: Fundamentals (1-10)

1. What is Node.js?

Node.js is an open-source, cross-platform JavaScript runtime environment built on Chrome's V8 JavaScript engine. It allows developers to run JavaScript on the server-side, enabling the development of scalable network applications.

Example:

```
// Simple Node.js server
const http = require('http');

const server = http.createServer((req, res) => {
  res.writeHead(200, {'Content-Type': 'text/plain'});
  res.end('Hello World from Node.js!');
});

server.listen(3000, () => {
  console.log('Server running on port 3000');
});
```

2. What is the Event Loop in Node.js?

The Event Loop is the mechanism that allows Node.js to perform non-blocking I/O operations despite JavaScript being single-threaded. It has phases: timers, pending callbacks, idle/prepare, poll, check, and close callbacks.

Example:

```
console.log('Start');

setTimeout(() => {
  console.log('Timeout');
}, 0);

setImmediate(() => {
  console.log('Immediate');
});
```

```
console.log('End');
```

```
// Output: Start, End, Immediate, Timeout
```

3. What is NPM and what is its purpose?

NPM (Node Package Manager) is the default package manager for Node.js that allows you to install, share, and manage dependencies.

Example:

```
# Initialize a new project  
npm init -y
```

```
# Install a package  
npm install express
```

```
# Install as dev dependency  
npm install --save-dev nodemon
```

```
# Install globally  
npm install -g pm2
```

4. What is the difference between package.json and package-lock.json?

`package.json` lists dependencies with version ranges, while `package-lock.json` locks exact versions for consistency.

Example package.json:

```
{  
  "name": "myapp",  
  "version": "1.0.0",  
  "dependencies": {  
    "express": "^4.18.0"  
  }  
}
```

5. What is module.exports and exports?

Both are used to export code from modules. `module.exports` is the actual object returned, while `exports` is a reference to it.

Example:

```
// math.js
module.exports.add = (a, b) => a + b;
exports.subtract = (a, b) => a - b;

// app.js
const math = require('./math');
console.log(math.add(5, 3)); // 8
console.log(math.subtract(5, 3)); // 2
```

6. What are built-in modules in Node.js?

Core modules that come pre-installed with Node.js like fs, http, path, os, events, stream, etc.

Example:

```
const fs = require('fs');
const path = require('path');
const os = require('os');

// File operations
fs.readFile('file.txt', 'utf8', (err, data) => {
  if (err) throw err;
  console.log(data);
});

// Path operations
const fullPath = path.join(__dirname, 'files', 'data.txt');

// OS info
console.log('Platform:', os.platform());
console.log('CPU cores:', os.cpus().length);
```

7. What is the difference between CommonJS and ES6 modules?

CommonJS uses `require()/module.exports` (synchronous), while ES6 uses `import/export` (can be asynchronous).

Example:

```
// CommonJS
const express = require('express');
module.exports = router;

// ES6 (need "type": "module" in package.json or .mjs extension)
import express from 'express';
export default router;
```

8. What is `__dirname` and `__filename`?

`__dirname` gives the directory path of the current file, `__filename` gives the full file path.

Example:

```
console.log(__dirname); // /Users/username/project
console.log(__filename); // /Users/username/project/app.js

const path = require('path');
const filePath = path.join(__dirname, 'data', 'users.json');
```

9. What is the global object in Node.js?

The global object is available everywhere and provides global variables and functions.

Example:

```
console.log(global);

// Global variables
global.myVar = 'Hello';
console.log(myVar); // Hello (not recommended)

// Available globally
console.log(__dirname);
console.log(process.env);
console.log(Buffer);
```

10. What is the difference between Node.js and Browser JavaScript?

Node.js has access to file system, no DOM, global object instead of window, and additional APIs.

Example:

```
// Node.js only
const fs = require('fs');
fs.writeFileSync('test.txt', 'Hello Node');

// Browser only
// document.getElementById('btn').addEventListener('click', fn);
// window.location.href = 'https://example.com';
```

Section 2: Asynchronous Programming (11-20)

11. What are Callbacks in Node.js?

Functions passed as arguments to be executed after an operation completes.

Example:

```
const fs = require('fs');

fs.readFile('data.txt', 'utf8', (err, data) => {
  if (err) {
    console.error('Error:', err);
    return;
  }
  console.log('File content:', data);
});

// Custom callback
function fetchUser(id, callback) {
  setTimeout(() => {
    callback(null, { id, name: 'John' });
  }, 1000);
}

fetchUser(1, (err, user) => {
  console.log(user);
});
```

12. What is callback hell and how to solve it?

Nested callbacks that make code hard to read. Solve with Promises or async/await.

Example:

```
// Callback hell
getData((err, data) => {
  getMoreData(data, (err, moreData) => {
    getEvenMore(moreData, (err, evenMore) => {
      // Pyramid of doom
    });
  });
});

// Solution with async/await
async function fetchAllData() {
  const data = await getData();
```

```

const moreData = await getMoreData(data);
const evenMore = await getEvenMore(moreData);
return evenMore;
}

```

13. What are Promises in Node.js?

Objects representing eventual completion or failure of async operations.

Example:

```

const promise = new Promise((resolve, reject) => {
  setTimeout(() => {
    const success = true;
    if (success) {
      resolve({ data: 'Success!' });
    } else {
      reject(new Error('Failed'));
    }
  }, 1000);
});

```

```

promise
  .then(result => console.log(result))
  .catch(error => console.error(error))
  .finally(() => console.log('Done'));

```

14. What is async/await?

Syntactic sugar for Promises that makes async code look synchronous.

Example:

```

// Without async/await
function getUser() {
  return fetch('/api/user')
    .then(res => res.json())
    .then(data => data);
}

```

```

// With async/await
async function getUser() {
  try {
    const res = await fetch('/api/user');
    const data = await res.json();
    return data;
  }
}

```

```
} catch (error) {  
  console.error('Error:', error);  
}  
}
```

15. What is Promise.all() and when to use it?

Executes multiple promises in parallel and waits for all to complete.

Example:

```
const promise1 = fetch('/api/users');  
const promise2 = fetch('/api/posts');  
const promise3 = fetch('/api/comments');  
  
Promise.all([promise1, promise2, promise3])  
  .then(([users, posts, comments]) => {  
    console.log('All data loaded');  
  })  
  .catch(error => {  
    console.error('One failed:', error);  
  });
```

16. What is the difference between Promise.all() and Promise.race()?

`Promise.all()` waits for all promises, `Promise.race()` returns when the first one completes.

Example:

```
// Promise.all - waits for all  
Promise.all([promise1, promise2, promise3])  
  .then(results => console.log('All done:', results));  
  
// Promise.race - first to finish wins  
Promise.race([promise1, promise2, promise3])  
  .then(result => console.log('First done:', result));
```

17. What is Promise.allSettled()?

Returns when all promises settle (resolve or reject), regardless of outcome.

Example:

```
const promises = [  
  Promise.resolve('Success 1'),
```

```

    Promise.reject('Error'),
    Promise.resolve('Success 2')
  ];

  Promise.allSettled(promises)
    .then(results => {
      results.forEach(result => {
        if (result.status === 'fulfilled') {
          console.log('Success:', result.value);
        } else {
          console.log('Failed:', result.reason);
        }
      });
    });
  });

```

18. What is `setImmediate()` vs `setTimeout()`?

`setImmediate()` executes in check phase, `setTimeout()` in timers phase of event loop.

Example:

```

setTimeout(() => {
  console.log('Timeout');
}, 0);

setImmediate(() => {
  console.log('Immediate');
});

// Output order depends on context
// In I/O cycle: Immediate, Timeout

```

19. What is `process.nextTick()`?

Executes callback after current operation completes, before event loop continues.

Example:

```

console.log('Start');

process.nextTick(() => {
  console.log('Next tick');
});

Promise.resolve().then(() => {
  console.log('Promise');
});

```



```
});  
  
console.log('End');  
  
// Output: Start, End, Next tick, Promise
```

20. How do you handle multiple async operations sequentially?

Use async/await in a loop or chain promises.

Example:

```
// Sequential execution  
async function processItems(items) {  
  for (const item of items) {  
    await processItem(item);  
  }  
}  
  
// Or using reduce  
async function processSequential(items) {  
  return items.reduce(async (promise, item) => {  
    await promise;  
    return processItem(item);  
  }, Promise.resolve());  
}
```

Section 3: Streams and Buffers (21-25)

21. What are Streams in Node.js?

Objects for reading or writing data in chunks. Types: Readable, Writable, Duplex, Transform.

Example:

```
const fs = require('fs');  
  
// Reading large file with stream  
const readStream = fs.createReadStream('large.txt', 'utf8');  
const writeStream = fs.createWriteStream('output.txt');  
  
readStream.on('data', (chunk) => {  
  console.log('Chunk received:', chunk.length);  
  writeStream.write(chunk);  
});
```

```
readStream.on('end', () => {  
  console.log('Reading completed');  
  writeStream.end();  
});
```

22. What is piping in streams?

Method to connect readable stream output to writable stream input.

Example:

```
const fs = require('fs');  
const zlib = require('zlib');  
  
// Pipe file through gzip compression  
fs.createReadStream('input.txt')  
  .pipe(zlib.createGzip())  
  .pipe(fs.createWriteStream('input.txt.gz'));  
  
// Chain multiple pipes  
fs.createReadStream('file.txt')  
  .pipe(transformStream)  
  .pipe(anotherTransform)  
  .pipe(fs.createWriteStream('output.txt'));
```

23. What is a Buffer in Node.js?

Temporary storage for binary data outside V8 heap.

Example:

```
// Create buffers  
const buf1 = Buffer.from('Hello');  
const buf2 = Buffer.alloc(10);  
const buf3 = Buffer.allocUnsafe(10);  
  
console.log(buf1); // <Buffer 48 65 6c 6c 6f>  
console.log(buf1.toString()); // Hello  
  
// Buffer operations  
const buf = Buffer.from('Hello World');  
console.log(buf.length); // 11  
console.log(buf.toString('hex')); // 48656c6c6f20576f726c64
```

24. How do you create a custom readable stream?

Extend the Readable class and implement the `_read` method.

Example:

```
const { Readable } = require('stream');

class MyReadable extends Readable {
  constructor(options) {
    super(options);
    this.counter = 0;
  }

  _read() {
    this.counter++;
    if (this.counter > 5) {
      this.push(null); // End stream
    } else {
      this.push(`Data ${this.counter}\n`);
    }
  }
}

const readable = new MyReadable();
readable.pipe(process.stdout);
```

25. How do you create a custom writable stream?

Extend the Writable class and implement the `_write` method.

Example:

```
const { Writable } = require('stream');

class MyWritable extends Writable {
  _write(chunk, encoding, callback) {
    console.log('Writing:', chunk.toString());
    callback();
  }
}

const writable = new MyWritable();
writable.write('Hello ');
writable.write('World!');
writable.end();
```

Section 4: Express.js (26-35)

26. What is Express.js?

Minimal and flexible Node.js web framework for building web applications and APIs.

Example:

```
const express = require('express');
const app = express();

app.get('/', (req, res) => {
  res.send('Hello Express!');
});

app.post('/users', (req, res) => {
  res.json({ message: 'User created' });
});

app.listen(3000, () => {
  console.log('Server running on port 3000');
});
```

27. What is middleware in Express?

Functions with access to req, res, and next that execute in sequence.

Example:

```
const express = require('express');
const app = express();

// Application-level middleware
app.use((req, res, next) => {
  console.log(`${req.method} ${req.url}`);
  next();
});

// Route-specific middleware
app.get('/users', authenticate, (req, res) => {
  res.json({ users: [] });
});

function authenticate(req, res, next) {
  if (req.headers.authorization) {
    next();
  } else {
    res.status(401).json({ error: 'Unauthorized' });
  }
}
```

```
}
```

28. What are different types of middleware?

Application-level, router-level, error-handling, built-in, and third-party middleware.

Example:

```
const express = require('express');
const app = express();

// Built-in middleware
app.use(express.json());
app.use(express.urlencoded({ extended: true }));
app.use(express.static('public'));

// Third-party middleware
const cors = require('cors');
const morgan = require('morgan');
app.use(cors());
app.use(morgan('dev'));

// Error-handling middleware (4 parameters)
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).send('Something broke!');
});
```

29. How do you handle routing in Express?

Using app.METHOD() or express.Router().

Example:

```
const express = require('express');
const app = express();
const router = express.Router();

// Basic routing
app.get('/users', (req, res) => {
  res.json({ users: [] });
});

// Route parameters
app.get('/users/:id', (req, res) => {
  res.json({ id: req.params.id });
});
```

```
});

// Using Router
router.get('/', (req, res) => {
  res.send('Products list');
});

router.post('/', (req, res) => {
  res.send('Create product');
});

app.use('/products', router);
```

30. How do you handle request body in Express?

Using body-parser middleware or built-in `express.json()`.

Example:

```
const express = require('express');
const app = express();

app.use(express.json()); // Parse JSON
app.use(express.urlencoded({ extended: true })); // Parse URL-encoded

app.post('/users', (req, res) => {
  const { name, email } = req.body;
  res.json({
    message: 'User created',
    user: { name, email }
  });
});
```

31. How do you serve static files in Express?

Using `express.static()` middleware.

Example:

```
const express = require('express');
const path = require('path');
const app = express();

// Serve static files from 'public' directory
app.use(express.static('public'));
```

```
// Serve from multiple directories
app.use('/static', express.static('public'));
app.use('/files', express.static('uploads'));

// With options
app.use(express.static('public', {
  maxAge: '1d',
  etag: false
}));
```

32. How do you handle errors in Express?

Using error-handling middleware with 4 parameters.

Example:

```
const express = require('express');
const app = express();

// Route that throws error
app.get('/error', (req, res, next) => {
  const err = new Error('Something went wrong');
  err.status = 500;
  next(err); // Pass to error handler
});

// Async error handling
app.get('/async-error', async (req, res, next) => {
  try {
    await someAsyncOperation();
    res.send('Success');
  } catch (error) {
    next(error);
  }
});

// Error handling middleware
app.use((err, req, res, next) => {
  res.status(err.status || 500);
  res.json({
    error: {
      message: err.message
    }
  });
});
```

33. What is the difference between `res.send()` and `res.json()`?

`res.send()` can send any type, `res.json()` converts to JSON and sets Content-Type.

Example:

```
app.get('/send', (req, res) => {
  res.send('Hello'); // text/html
  res.send({ name: 'John' }); // application/json
  res.send(Buffer.from('Binary'));
});

app.get('/json', (req, res) => {
  res.json({ name: 'John' }); // Always application/json
  // res.json() also handles null and converts to JSON string
});
```

34. How do you implement authentication in Express?

Using middleware for token verification.

Example:

```
const jwt = require('jsonwebtoken');

// Login route
app.post('/login', (req, res) => {
  const { username, password } = req.body;

  // Verify credentials
  if (username === 'admin' && password === 'pass') {
    const token = jwt.sign({ username }, 'secret', { expiresIn: '1h' });
    res.json({ token });
  } else {
    res.status(401).json({ error: 'Invalid credentials' });
  }
});

// Authentication middleware
function authenticate(req, res, next) {
  const token = req.headers.authorization?.split(' ')[1];

  if (!token) {
    return res.status(401).json({ error: 'No token provided' });
  }

  try {
```



```

    const decoded = jwt.verify(token, 'secret');
    req.user = decoded;
    next();
  } catch (error) {
    res.status(401).json({ error: 'Invalid token' });
  }
}

// Protected route
app.get('/protected', authenticate, (req, res) => {
  res.json({ message: 'Access granted', user: req.user });
});

```

35. How do you implement rate limiting in Express?

Using express-rate-limit middleware.

Example:

```

const rateLimit = require('express-rate-limit');

const limiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 100, // Limit each IP to 100 requests per windowMs
  message: 'Too many requests from this IP'
});

// Apply to all routes
app.use(limiter);

// Apply to specific routes
const loginLimiter = rateLimit({
  windowMs: 60 * 60 * 1000, // 1 hour
  max: 5,
  message: 'Too many login attempts'
});

app.post('/login', loginLimiter, (req, res) => {
  // Login logic
});

```

Section 5: File System (36-40)

36. How do you read a file in Node.js?

Using `fs.readFile()` (async) or `fs.readFileSync()` (sync).

Example:

```
const fs = require('fs');

// Asynchronous
fs.readFile('file.txt', 'utf8', (err, data) => {
  if (err) throw err;
  console.log(data);
});

// Synchronous
try {
  const data = fs.readFileSync('file.txt', 'utf8');
  console.log(data);
} catch (err) {
  console.error(err);
}

// Using Promises
const fsPromises = require('fs').promises;
async function readFile() {
  const data = await fsPromises.readFile('file.txt', 'utf8');
  console.log(data);
}
```

37. How do you write to a file in Node.js?

Using `fs.writeFile()` or `fs.appendFile()`.

Example:

```
const fs = require('fs');

// Write (overwrite)
fs.writeFile('output.txt', 'Hello World', (err) => {
  if (err) throw err;
  console.log('File written');
});

// Append
fs.appendFile('log.txt', 'New log entry\n', (err) => {
  if (err) throw err;
  console.log('Log appended');
});
```

```
// Using Promises
const fsPromises = require('fs').promises;
async function writeFile() {
  await fsPromises.writeFile('output.txt', 'Hello');
  console.log('File written');
}
```

38. How do you check if a file exists?

Using `fs.existsSync()` or `fs.access()`.

Example:

```
const fs = require('fs');

// Synchronous
if (fs.existsSync('file.txt')) {
  console.log('File exists');
}

// Asynchronous
fs.access('file.txt', fs.constants.F_OK, (err) => {
  if (err) {
    console.log('File does not exist');
  } else {
    console.log('File exists');
  }
});

// Using Promises
const fsPromises = require('fs').promises;
async function checkFile() {
  try {
    await fsPromises.access('file.txt');
    console.log('File exists');
  } catch {
    console.log('File does not exist');
  }
}
```

39. How do you delete a file in Node.js?

Using `fs.unlink()` or `fs.unlinkSync()`.

Example:

```

const fs = require('fs');

// Asynchronous
fs.unlink('file.txt', (err) => {
  if (err) throw err;
  console.log('File deleted');
});

// Synchronous
try {
  fs.unlinkSync('file.txt');
  console.log('File deleted');
} catch (err) {
  console.error(err);
}

// Using Promises
const fsPromises = require('fs').promises;
async function deleteFile() {
  await fsPromises.unlink('file.txt');
  console.log('File deleted');
}

```

40. How do you work with directories?

Using fs methods for directory operations.

Example:

```

const fs = require('fs');
const path = require('path');

// Create directory
fs.mkdir('newdir', { recursive: true }, (err) => {
  if (err) throw err;
  console.log('Directory created');
});

// Read directory
fs.readdir('./mydir', (err, files) => {
  if (err) throw err;
  files.forEach(file => {
    console.log(file);
  });
});

// Remove directory

```

```

fs.rmdir('mydir', { recursive: true }, (err) => {
  if (err) throw err;
  console.log('Directory removed');
});

// Check if directory exists
if (fs.existsSync('./mydir') && fs.statSync('./mydir').isDirectory()) {
  console.log('Directory exists');
}

```

Section 6: Events and Error Handling (41-45)

41. What is EventEmitter in Node.js?

Class that allows objects to emit and listen to events.

Example:

```

const EventEmitter = require('events');

class MyEmitter extends EventEmitter {}

const emitter = new MyEmitter();

// Listen to event
emitter.on('event', (data) => {
  console.log('Event received:', data);
});

// Emit event
emitter.emit('event', { message: 'Hello' });

// Listen once
emitter.once('single', () => {
  console.log('This runs only once');
});

emitter.emit('single');
emitter.emit('single'); // Won't trigger

```

42. How do you handle errors in EventEmitter?

Using the 'error' event handler.

Example:

```
const EventEmitter = require('events');
const emitter = new EventEmitter();

// Error handler
emitter.on('error', (err) => {
  console.error('Error occurred:', err.message);
});

// Emit error
emitter.emit('error', new Error('Something failed'));

// Without error handler, it will crash
// emitter.emit('error', new Error('Crash!'));
```

43. What is `process.on('uncaughtException')`?

Global error handler for uncaught exceptions.

Example:

```
process.on('uncaughtException', (err) => {
  console.error('Uncaught Exception:', err);
  // Log error and exit gracefully
  process.exit(1);
});

// This error will be caught
setTimeout(() => {
  throw new Error('Oops!');
}, 1000);
```

44. What is `process.on('unhandledRejection')`?

Global handler for unhandled promise rejections.

Example:

```
process.on('unhandledRejection', (reason, promise) => {
  console.error('Unhandled Rejection at:', promise);
  console.error('Reason:', reason);
  // Log and exit
  process.exit(1);
});

// This rejection will be caught
Promise.reject(new Error('Rejected!'));
```

```
// Better way - always handle promises
async function safeFetch() {
  try {
    await fetch('url');
  } catch (error) {
    console.error('Error:', error);
  }
}
```

45. How do you create custom error classes?

Extend the Error class.

Example:

```
class ValidationError extends Error {
  constructor(message) {
    super(message);
    this.name = 'ValidationError';
    this.statusCode = 400;
  }
}
```

```
class NotFoundError extends Error {
  constructor(message) {
    super(message);
    this.name = 'NotFoundError';
    this.statusCode = 404;
  }
}
```

```
// Usage
function validateUser(user) {
  if (!user.email) {
    throw new ValidationError('Email is required');
  }
}
```

```
try {
  validateUser({});
} catch (error) {
  if (error instanceof ValidationError) {
    console.log('Validation failed:', error.message);
  }
}
```

Section 7: Database & APIs (46-50)

46. How do you connect to MongoDB using Mongoose?

Using `mongoose.connect()` method.

Example:

```
const mongoose = require('mongoose');

// Connect to MongoDB
mongoose.connect('mongodb://localhost:27017/mydb', {
  useNewUrlParser: true,
  useUnifiedTopology: true
});

const db = mongoose.connection;

db.on('error', console.error.bind(console, 'connection error:'));
db.once('open', () => {
  console.log('Connected to MongoDB');
});

// Define schema
const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  age: Number,
  createdAt: { type: Date, default: Date.now }
});

// Create model
const User = mongoose.model('User', userSchema);

// Create document
const user = new User({
  name: 'John',
  email: 'john@example.com',
  age: 30
});

user.save()
  .then(() => console.log('User saved'))
  .catch(err => console.error(err));
```

47. How do you perform CRUD operations with MongoDB?

Using Mongoose model methods.

Example:

```
const User = require('./models/User');

// CREATE
async function createUser() {
  const user = await User.create({
    name: 'John',
    email: 'john@example.com'
  });
  console.log('Created:', user);
}

// READ
async function getUsers() {
  const users = await User.find();
  const oneUser = await User.findById(id);
  const filtered = await User.find({ age: { $gte: 18 } });
  return users;
}

// UPDATE
async function updateUser(id) {
  const user = await User.findByIdAndUpdate(
    id,
    { name: 'Jane' },
    { new: true } // Return updated document
  );
  console.log('Updated:', user);
}

// DELETE
async function deleteUser(id) {
  await User.findByIdAndDelete(id);
  console.log('Deleted');
}
```

48. How do you create a REST API with Express?

Create routes for CRUD operations.

Example:

```
const express = require('express');
const app = express();
```

```
const User = require('./models/User');

app.use(express.json());

// GET all users
app.get('/api/users', async (req, res) => {
  try {
    const users = await User.find();
    res.json(users);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

// GET single user
app.get('/api/users/:id', async (req, res) => {
  try {
    const user = await User.findById(req.params.id);
    if (!user) {
      return res.status(404).json({ error: 'User not found' });
    }
    res.json(user);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

// POST create user
app.post('/api/users', async (req, res) => {
  try {
    const user = await User.create(req.body);
    res.status(201).json(user);
  } catch (error) {
    res.status(400).json({ error: error.message });
  }
});

// PUT update user
app.put('/api/users/:id', async (req, res) => {
  try {
    const user = await User.findByIdAndUpdate(
      req.params.id,
      req.body,
      { new: true, runValidators: true }
    );
    if (!user) {
      return res.status(404).json({ error: 'User not found' });
    }
  }
});
```

```

    res.json(user);
  } catch (error) {
    res.status(400).json({ error: error.message });
  }
});

// DELETE user
app.delete('/api/users/:id', async (req, res) => {
  try {
    const user = await User.findByIdAndDelete(req.params.id);
    if (!user) {
      return res.status(404).json({ error: 'User not found' });
    }
    res.json({ message: 'User deleted' });
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

app.listen(3000, () => console.log('Server running on port 3000'));

```

49. How do you implement JWT authentication?

Using jsonwebtoken package for token generation and verification.

Example:

```

const jwt = require('jsonwebtoken');
const bcrypt = require('bcrypt');
const User = require('./models/User');

const SECRET_KEY = process.env.JWT_SECRET || 'your-secret-key';

// Register user
app.post('/api/register', async (req, res) => {
  try {
    const { name, email, password } = req.body;

    // Hash password
    const hashedPassword = await bcrypt.hash(password, 10);

    // Create user
    const user = await User.create({
      name,
      email,
      password: hashedPassword
    });
  }
});

```

```

// Generate token
const token = jwt.sign(
  { userId: user._id, email: user.email },
  SECRET_KEY,
  { expiresIn: '24h' }
);

res.status(201).json({ token, user: { id: user._id, name, email } });
} catch (error) {
  res.status(400).json({ error: error.message });
}
});

// Login user
app.post('/api/login', async (req, res) => {
  try {
    const { email, password } = req.body;

    // Find user
    const user = await User.findOne({ email });
    if (!user) {
      return res.status(401).json({ error: 'Invalid credentials' });
    }

    // Verify password
    const isValidPassword = await bcrypt.compare(password, user.password);
    if (!isValidPassword) {
      return res.status(401).json({ error: 'Invalid credentials' });
    }

    // Generate token
    const token = jwt.sign(
      { userId: user._id, email: user.email },
      SECRET_KEY,
      { expiresIn: '24h' }
    );

    res.json({ token, user: { id: user._id, name: user.name, email } });
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

// Auth middleware
function authenticate(req, res, next) {
  const token = req.headers.authorization?.replace('Bearer ', '');

```

```

if (!token) {
  return res.status(401).json({ error: 'No token provided' });
}

try {
  const decoded = jwt.verify(token, SECRET_KEY);
  req.user = decoded;
  next();
} catch (error) {
  res.status(401).json({ error: 'Invalid token' });
}
}

// Protected route
app.get('/api/profile', authenticate, async (req, res) => {
  try {
    const user = await User.findById(req.user.userId).select('-password');
    res.json(user);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

```

50. How do you handle file uploads in Node.js?

Using multer middleware for handling multipart/form-data.

Example:

```

const express = require('express');
const multer = require('multer');
const path = require('path');
const app = express();

// Configure storage
const storage = multer.diskStorage({
  destination: (req, file, cb) => {
    cb(null, 'uploads/');
  },
  filename: (req, file, cb) => {
    const uniqueSuffix = Date.now() + '-' + Math.round(Math.random() * 1E9);
    cb(null, file.fieldname + '-' + uniqueSuffix + path.extname(file.originalname));
  }
});

// File filter
const fileFilter = (req, file, cb) => {

```

```

const allowedTypes = /jpeg|jpg|png|gif/;
const extname = allowedTypes.test(path.extname(file.originalName).toLowerCase());
const mimetype = allowedTypes.test(file.mimetype);

if (extname && mimetype) {
  cb(null, true);
} else {
  cb(new Error('Only image files are allowed'));
}
};

// Initialize upload
const upload = multer({
  storage: storage,
  limits: { fileSize: 1024 * 1024 * 5 }, // 5MB
  fileFilter: fileFilter
});

// Single file upload
app.post('/upload', upload.single('image'), (req, res) => {
  if (!req.file) {
    return res.status(400).json({ error: 'No file uploaded' });
  }
  res.json({
    message: 'File uploaded successfully',
    file: {
      filename: req.file.filename,
      path: req.file.path,
      size: req.file.size
    }
  });
});

// Multiple files upload
app.post('/upload-multiple', upload.array('images', 5), (req, res) => {
  if (!req.files || req.files.length === 0) {
    return res.status(400).json({ error: 'No files uploaded' });
  }
  res.json({
    message: 'Files uploaded successfully',
    files: req.files.map(file => ({
      filename: file.filename,
      path: file.path,
      size: file.size
    }))
  });
});

```

```
// Error handling
app.use((err, req, res, next) => {
  if (err instanceof multer.MulterError) {
    return res.status(400).json({ error: err.message });
  }
  res.status(500).json({ error: err.message });
});
```

Section 8: Performance & Security (51-55)

51. What is clustering in Node.js?

Creating multiple worker processes to utilize all CPU cores.

Example:

```
const cluster = require('cluster');
const http = require('http');
const os = require('os');

if (cluster.isMaster) {
  const numCPUs = os.cpus().length;

  console.log(`Master ${process.pid} is running`);

  // Fork workers
  for (let i = 0; i < numCPUs; i++) {
    cluster.fork();
  }

  // Handle worker death
  cluster.on('exit', (worker, code, signal) => {
    console.log(`Worker ${worker.process.pid} died`);
    console.log('Starting a new worker');
    cluster.fork();
  });
} else {
  // Workers can share TCP connection
  http.createServer((req, res) => {
    res.writeHead(200);
    res.end(`Worker ${process.pid} handled the request\n`);
  }).listen(8000);

  console.log(`Worker ${process.pid} started`);
}
```

52. How do you implement caching in Node.js?

Using in-memory cache or Redis.

Example:

```
const redis = require('redis');
const client = redis.createClient();

// Simple in-memory cache
const cache = {};

// Cache middleware
function cacheMiddleware(duration) {
  return (req, res, next) => {
    const key = req.originalUrl || req.url;
    const cachedResponse = cache[key];

    if (cachedResponse) {
      console.log('Cache hit');
      return res.send(cachedResponse);
    }

    // Store original send function
    res.originalSend = res.send;

    // Override send function
    res.send = function(body) {
      cache[key] = body;
      setTimeout(() => delete cache[key], duration * 1000);
      res.originalSend(body);
    };

    next();
  };
}

// Using Redis cache
async function getCacheData(key) {
  return new Promise((resolve, reject) => {
    client.get(key, (err, data) => {
      if (err) reject(err);
      if (data) {
        resolve(JSON.parse(data));
      } else {
        resolve(null);
      }
    });
  });
}
```



```

    }
  });
});
}

```

```

async function setCachedData(key, data, expiration = 3600) {
  return new Promise((resolve, reject) => {
    client.setex(key, expiration, JSON.stringify(data), (err) => {
      if (err) reject(err);
      resolve();
    });
  });
}

```

```

// Usage
app.get('/api/users', cacheMiddleware(60), async (req, res) => {
  const users = await User.find();
  res.json(users);
});

```

53. How do you prevent SQL injection in Node.js?

Using parameterized queries and ORM/ODM.

Example:

```

const mysql = require('mysql2/promise');

// BAD - Vulnerable to SQL injection
app.get('/user', async (req, res) => {
  const userId = req.query.id;
  const query = `SELECT * FROM users WHERE id = ${userId}`;
  // Attacker can send: ?id=1 OR 1=1
});

// GOOD - Using parameterized queries
app.get('/user', async (req, res) => {
  const connection = await mysql.createConnection(config);
  const userId = req.query.id;

  const [rows] = await connection.execute(
    'SELECT * FROM users WHERE id = ?',
    [userId]
  );

  res.json(rows);
});

```

```

// GOOD - Using ORM (Mongoose)
app.get('/user/:id', async (req, res) => {
  const user = await User.findById(req.params.id);
  res.json(user);
});

// Input validation
const { body, validationResult } = require('express-validator');

app.post('/user',
  body('email').isEmail(),
  body('age').isInt({ min: 0, max: 120 }),
  async (req, res) => {
    const errors = validationResult(req);
    if (!errors.isEmpty()) {
      return res.status(400).json({ errors: errors.array() });
    }

    const user = await User.create(req.body);
    res.json(user);
  }
);

```

54. How do you implement HTTPS in Node.js?

Using https module with SSL certificates.

Example:

```

const https = require('https');
const fs = require('fs');
const express = require('express');

const app = express();

// Read SSL certificate files
const options = {
  key: fs.readFileSync('private-key.pem'),
  cert: fs.readFileSync('certificate.pem')
};

app.get('/', (req, res) => {
  res.send('Secure HTTPS connection');
});

// Create HTTPS server

```

```

https.createServer(options, app).listen(443, () => {
  console.log('HTTPS Server running on port 443');
});

// Redirect HTTP to HTTPS
const http = require('http');
http.createServer((req, res) => {
  res.writeHead(301, { Location: `https://${req.headers.host}${req.url}` });
  res.end();
}).listen(80);

```

55. How do you implement helmet for security?

Using helmet middleware to set security headers.

Example:

```

const express = require('express');
const helmet = require('helmet');
const app = express();

// Use helmet with default settings
app.use(helmet());

// Or configure specific headers
app.use(
  helmet({
    contentSecurityPolicy: {
      directives: {
        defaultSrc: ["'self'"],
        styleSrc: ["'self'", "unsafe-inline"],
        scriptSrc: ["'self'", "trusted-cdn.com"]
      }
    },
    hsts: {
      maxAge: 31536000,
      includeSubDomains: true,
      preload: true
    }
  })
);

// Individual helmet middleware
app.use(helmet.hidePoweredBy()); // Hide X-Powered-By header
app.use(helmet.frameguard({ action: 'deny' })); // Prevent clickjacking
app.use(helmet.xssFilter()); // XSS protection
app.use(helmet.noSniff()); // Prevent MIME sniffing

```

Bonus Questions (56-60)

56. What is Worker Threads in Node.js?

Allows running JavaScript operations in parallel threads.

Example:

```
const { Worker, isMainThread, parentPort } = require('worker_threads');
```

```
if (isMainThread) {  
  // Main thread  
  const worker = new Worker(__filename);  
  
  worker.on('message', (result) => {  
    console.log('Result from worker:', result);  
  });  
  
  worker.postMessage({ num: 42 });  
  
} else {
```

```
  // Worker thread  
  parentPort.on('message', (data) => {  
    const result = data.num * 2;  
    parentPort.postMessage(result);  
  });  
}
```

```
// Heavy computation example  
function fibonacci(n) {  
  if (n <= 1) return n;  
  return fibonacci(n - 1) + fibonacci(n - 2);  
}
```

```
// In main thread  
const worker = new Worker(`  
  const { parentPort } = require('worker_threads');  
  parentPort.on('message', (n) => {  
    function fib(n) {  
      if (n <= 1) return n;  
      return fib(n - 1) + fib(n - 2);  
    }  
    parentPort.postMessage(fib(n));  
  });  
`, { eval: true });
```

```
worker.postMessage(40);
worker.on('message', (result) => {
  console.log('Fibonacci result:', result);
});
```

57. How do you implement WebSockets in Node.js?

Using socket.io or ws library for real-time communication.

Example:

```
const express = require('express');
const http = require('http');
const socketio = require('socket.io');

const app = express();
const server = http.createServer(app);
const io = socketio(server);

// Handle connections
io.on('connection', (socket) => {
  console.log('User connected:', socket.id);

  // Listen for messages
  socket.on('chat message', (msg) => {
    console.log('Message:', msg);
    // Broadcast to all clients
    io.emit('chat message', msg);
  });

  // Join room
  socket.on('join room', (room) => {
    socket.join(room);
    socket.to(room).emit('user joined', socket.id);
  });

  // Private message
  socket.on('private message', ({ to, message }) => {
    socket.to(to).emit('private message', {
      from: socket.id,
      message
    });
  });

  // Handle disconnect
  socket.on('disconnect', () => {
```

```

    console.log('User disconnected:', socket.id);
  });
});

server.listen(3000, () => {
  console.log('Server running on port 3000');
});

// Client-side example
/*
<script src="/socket.io/socket.io.js"></script>
<script>
  const socket = io();

  socket.on('connect', () => {
    console.log('Connected to server');
  });

  socket.emit('chat message', 'Hello!');

  socket.on('chat message', (msg) => {
    console.log('Received:', msg);
  });
</script>
*/

```

58. How do you implement pagination in APIs?

Using limit and skip parameters.

Example:

```

// Pagination middleware
async function paginate(model, req) {
  const page = parseInt(req.query.page) || 1;
  const limit = parseInt(req.query.limit) || 10;
  const skip = (page - 1) * limit;

  const total = await model.countDocuments();
  const data = await model.find()
    .limit(limit)
    .skip(skip)
    .sort({ createdAt: -1 });

  return {
    data,
    pagination: {

```

```

    currentPage: page,
    totalPages: Math.ceil(total / limit),
    totalItems: total,
    itemsPerPage: limit,
    hasNextPage: page * limit < total,
    hasPrevPage: page > 1
  }
};
}

// API endpoint
app.get('/api/users', async (req, res) => {
  try {
    const result = await paginate(User, req);
    res.json(result);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

// Advanced pagination with filters
app.get('/api/products', async (req, res) => {
  const page = parseInt(req.query.page) || 1;
  const limit = parseInt(req.query.limit) || 10;
  const skip = (page - 1) * limit;

  // Build filter
  const filter = {};
  if (req.query.category) {
    filter.category = req.query.category;
  }
  if (req.query.minPrice) {
    filter.price = { $gte: parseFloat(req.query.minPrice) };
  }

  const total = await Product.countDocuments(filter);
  const products = await Product.find(filter)
    .limit(limit)
    .skip(skip)
    .sort({ createdAt: -1 });

  res.json({
    products,
    page,
    totalPages: Math.ceil(total / limit),
    totalProducts: total
  });
});

```

59. How do you implement logging in Node.js?

Using winston or morgan for logging.

Example:

```
const winston = require('winston');
const morgan = require('morgan');

// Configure Winston logger
const logger = winston.createLogger({
  level: 'info',
  format: winston.format.combine(
    winston.format.timestamp(),
    winston.format.json()
  ),
  transports: [
    new winston.transports.File({ filename: 'error.log', level: 'error' }),
    new winston.transports.File({ filename: 'combined.log' }),
    new winston.transports.Console({
      format: winston.format.simple()
    })
  ]
});

// Morgan for HTTP logging
app.use(morgan('combined', {
  stream: {
    write: (message) => logger.info(message.trim())
  }
}));

// Custom logging
logger.info('Application started');
logger.error('An error occurred', { error: err.message });
logger.warn('Warning message');
logger.debug('Debug information');

// Request logging middleware
app.use((req, res, next) => {
  const start = Date.now();

  res.on('finish', () => {
    const duration = Date.now() - start;
    logger.info({
      method: req.method,
```



```

    url: req.url,
    status: res.statusCode,
    duration: `${duration}ms`,
    ip: req.ip
  });
});

  next();
});

```

60. How do you implement environment variables?

Using dotenv package for configuration management.

Example:

```

// .env file
/*
NODE_ENV=development
PORT=3000
DB_HOST=localhost
DB_PORT=27017
DB_NAME=myapp
JWT_SECRET=your-secret-key
API_KEY=your-api-key
*/

// Load environment variables
require('dotenv').config();

const express = require('express');
const app = express();

// Access environment variables
const PORT = process.env.PORT || 3000;
const DB_URL =
`mongodb://${process.env.DB_HOST}:${process.env.DB_PORT}/${process.env.DB_NAME}`
;

// Configuration object
const config = {
  env: process.env.NODE_ENV,
  port: process.env.PORT,
  database: {
    host: process.env.DB_HOST,
    port: process.env.DB_PORT,
    name: process.env.DB_NAME
  }
}

```

```

    },
    jwt: {
      secret: process.env.JWT_SECRET,
      expiresIn: '24h'
    }
  };

  // Environment-specific logic
  if (process.env.NODE_ENV === 'production') {
    // Production settings
    app.use(compression());
    app.use(helmet());
  } else {
    // Development settings
    app.use(morgan('dev'));
  }

  app.listen(PORT, () => {
    console.log(`Server running on port ${PORT}`);
    console.log(`Environment: ${process.env.NODE_ENV}`);
  });

  // Validation
  const requiredEnvVars = ['DB_HOST', 'JWT_SECRET', 'API_KEY'];
  requiredEnvVars.forEach((varName) => {
    if (!process.env[varName]) {
      throw new Error(`Environment variable ${varName} is required`);
    }
  });
});

```

Summary

This document covers 60 essential Node.js interview questions with practical code examples. Topics include:

- **Fundamentals** (1-10): Node.js basics, modules, NPM
- **Async Programming** (11-20): Callbacks, Promises, async/await
- **Streams & Buffers** (21-25): Data handling
- **Express.js** (26-35): Web framework, middleware, routing
- **File System** (36-40): File operations
- **Events & Errors** (41-45): Event handling, error management
- **Database & APIs** (46-50): MongoDB, REST, JWT, file uploads
- **Performance & Security** (51-55): Clustering, caching, security
- **Bonus Topics** (56-60): Worker threads, WebSockets, pagination, logging, environment variables

Interview Tips:

- Practice writing code examples
- Understand the "why" behind concepts
- Be ready to explain real-world use cases
- Know trade-offs between different approaches
- Stay updated with latest Node.js features